Hoja de repaso para la review — Sprint 2 (Arquitectura de Software)

Plataforma de Reservas de Servicios · Estado a 27 de octubre de 2026 · App: https://proyecto2026-2-5zoo.onrender.com/swagger-ui.html

En una frase

Backend REST **monolítico y modular** (Spring Boot 4.1.1 / Java 17 / PostgreSQL 16) para reservar servicios: un solo artefacto desplegable, dividido en módulos por dominio que se comunican solo por interfaces.

Estado hoy

Tema	Estado
Alcance	**15 de las 22 HU del Sprint 2** más las 6 del Sprint 1; 33 endpoints REST
Calidad	**658 pruebas** automáticas, cobertura ≈ 96 %, Sonar con Quality Gate OK (0 issues, 0 vulnerabilidades)
Despliegue	Render + GitHub Actions; la **base de datos pasó a Neon el 25-oct** (el plan gratuito de Render se agotó); Swagger activo
Pendiente	7 HU para el Sprint 3, proteger `main`, ratificar los ADR 004–007, mejoras de MFA

La arquitectura

Módulo	Qué hace
`identity`	Registro, login, sesiones, roles, MFA, eliminación de cuentas
`provider`	Proveedor y su negocio, antelación mínima de reserva
`service`	Servicios, catálogo y **motor de disponibilidad**
`resource`	Recursos (salas, equipos), horarios, desactivar y reactivar
`reservation`	Reservas y cancelaciones
`audit` · `common`	Registro de eventos · formato de error, configuración y filtros (transversales)

- **Dependencias en una sola dirección, sin ciclos:** `identity ← provider ← resource ← service ← reservation` (lo verifica un script).

- **Capas por módulo:** `controller` → `application` → `infrastructure` → `domain`.

- **Comunicación:** solo por interfaces de la capa `application` (`*Lookup`, `*Directory`) con objetos simples; **sin `@ManyToOne` entre módulos** (referencias por clave foránea); y **un evento síncrono** (`UserDeletionRequested`) para que `reservation` reaccione cuando `identity` elimina una cuenta.

Decisiones clave y por qué

Decisión	Por qué
Monolito modular, no microservicios	Equipo de 3 y un despliegue; los cortes por módulo dejan abierta la separación futura
JWT de 1 h + tabla de sesiones	Sin estado, pero revocable: el logout funciona de verdad
MFA TOTP obligatorio para administradores	Son el rol con más poder; clientes y proveedores lo tienen voluntario
Anti-overbooking **en la base de datos**	Una restricción `EXCLUDE` impide dos reservas confirmadas traslapadas del mismo recurso; una verificación en memoria no es atómica
Reservas con copias («snapshots») y `ON DELETE SET NULL`	Eliminar cuentas, servicios o negocios no borra ni bloquea el historial
Cancelación = estado + origen + motivo	Evita multiplicar estados (`CANCELADA` + `PROVEEDOR` / `CLIENTE` / `ELIMINACION_CUENTA`…)
Formato de error único con `traceId`	Cada fallo se puede buscar en los logs

Seguridad (OWASP): control de acceso por rol y por pertenencia, validación con límites, límites de intentos por IP real, cabeceras CSP/Referrer, CORS cerrado, tope de cuerpo de 64 KB, contenedor sin root, arranque que falla con claves de relleno, Dependabot. Es una **revisión con correcciones, no una prueba de penetración**.

CI/CD: GitHub → Actions (pruebas → Sonar → build → *deploy hook* a Render con *smoke test*) → contenedor Docker en Render + PostgreSQL en Neon.

Preguntas probables y respuestas cortas

Pregunta	Respuesta
¿Por qué monolito y no microservicios?	Alcance, equipo y costo; el diseño modular permite separar después

¿Cómo evitan que dos clientes reserven lo mismo?	Restricción de exclusión en PostgreSQL más bloqueo de fila; probado con concurrencia real
¿Qué pasa al eliminar un usuario con reservas?	Sus reservas futuras se cancelan en la misma transacción y el historial se conserva (copias de nombre, servicio y precio)
¿Por qué MFA solo para administradores?	Es el rol de mayor impacto; para los demás sería fricción sin un riesgo que lo justifique todavía
¿Cómo funciona el MFA?	TOTP (RFC 6238): servidor y celular calculan un HMAC del secreto compartido con el tiempo dividido en tramos de 30 s; no se comunican entre sí. El servidor acepta también el tramo anterior y el siguiente
¿Limitaciones?	Una sola instancia (límites de intentos en memoria), cold start del plan gratuito, anti-replay y cifrado del secreto MFA pendientes, `main` sin protección
¿Por qué pasaron de Render a Neon?	El plan gratuito de Render limita horas y expira la base a los 30 días; en Neon la base no vence
¿Por qué no hicieron las 22 HU?	Se priorizó el flujo completo de reserva; las 7 restantes son edición y mantenimiento (07, 10, 11, 12, 15, 21) y completar reserva (27, junto con Report)
¿Qué sigue en el Sprint 3?	Contenedores y orquestación, observabilidad (Grafana/Prometheus), refinamiento de APIs y las HU restantes

Documentos de apoyo (en el repositorio)

Si preguntan por…	Mira
Visión general y evidencias por entregable	informe-arquitectura-sprint-2.pdf · entregable-arquitectura-sprint-2.md
Decisiones y por qué	docs/arquitectura/adr/ (ADR 001–007)
Qué supusimos y a quién confirmar	decisiones-y-supuestos-sprint-2.md (S-01…S-32 y preguntas para Calidad)
Despliegue	diagrama-despliegue.png · guia-recuperar-despliegue.md
Endpoints, roles y DTO	referencia-api-sprint-2.md · errores-api-sprint-2.md
Seguridad	owasp-top10-sprint-2.md · ADR-004 (MFA)
Pruebas	resultados-pruebas-sprint-2.md · docs/postman/
Priorización de las HU	plan-de-trabajo-sprint-2.md §2
Estado vivo y bitácora	handoff-contexto.md

Si te preguntan algo que no sabes: «lo dejamos documentado como supuesto en `decisiones-y-supuestos-sprint-2.md` y lo confirmamos con Calidad/PO».